

Chương 8: Transformers

Môn: CS221.Q21.KHTN - Xử lý ngôn ngữ tự nhiên

Thành viên thực hiện:

Hà Thanh Phong - 24520024

Hà Xuân Thiện - 24520031

Trần Quang Trường - 24521901

Trường Đại học Công nghệ Thông tin (UIT)

Ngày 14 tháng 4 năm 2026

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

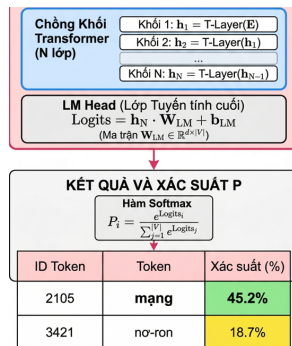
8.5. Khái niệm và Vai trò của "Head"

Khái niệm:

- "Head" là các **mạch neural**, được **tích hợp ở cuối** cấu trúc Transformer nhằm thực thi các **tác vụ chuyên biệt**.
- Đối với **Language Modeling Head**, mạng neural này hoạt động như một bộ dự đoán từ (word predictor).

Mục tiêu:

- Dựa trên **ngữ cảnh**, mô hình **gán xác suất** cho mọi từ tiềm năng, tạo thành một **phân phối** trên toàn bộ **từ điển V**.



Minh họa cơ chế dự đoán từ

8.5 Cấu trúc Đầu vào

Luồng dữ liệu (Data Flow):

- Tiếp nhận **vector đầu ra** h_N^L từ lớp Transformer cuối cùng tại token N .
- h_N^L được sử dụng làm **cơ sở suy diễn** từ vựng tại vị trí $N + 1$.
- **Figure 8.15** mô tả quy trình ánh xạ vector trạng thái ẩn $[1 \times d]$ thành **phân phối xác suất**.

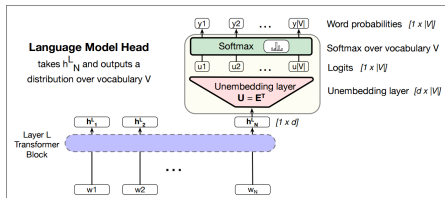


Figure 8.15 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer (h_N^L) to a probability distribution over words in the vocabulary V .

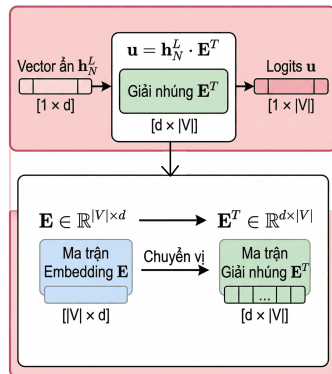
8.5 Phép chiếu Tuyến tính và Giải nhúng

1. Lớp Tuyến tính (Linear Layer):

- Chiếu vector ẩn h_N^L thành vector logit u .
- Biến đổi không gian từ $[1 \times d]$ sang không gian từ vựng $[1 \times |V|]$.

2. Kỹ thuật Weight Tying:

- Sử dụng ma trận chuyển vị E^T làm lớp Giải nhúng (Unembedding layer) để ánh xạ ngược về từ vựng.



Ánh xạ ngược qua ma trận chuyển vị E^T

8.5 Chuẩn hóa Xác suất và Sinh văn bản

Hàm Softmax:

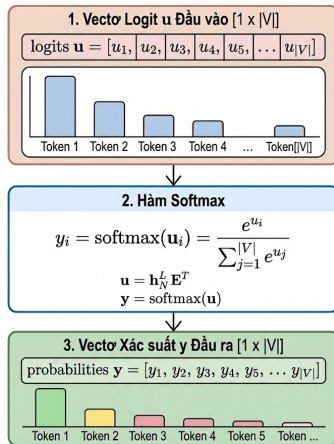
- Chuyển đổi logit u thành phân phối xác suất y .
- Hệ phương trình cốt lõi:

$$u = h_N^L E^T \quad (8.46)$$

$$y = \text{softmax}(u) \quad (8.47)$$

Quá trình Lấy mẫu (Sampling):

- Sinh văn bản bằng cách lấy mẫu một từ từ phân phối y .



Từ Logit đến xác suất

8.5 Kiến trúc Decoder-Only

Kiến trúc Xếp chồng (Stacked Architecture):

- Đầu vào mỗi lớp x_i^L kế thừa trực tiếp từ đầu ra h_i^{L-1} của lớp liền trước.

Thuật ngữ Decoder-Only:

- Định danh cho các mô hình ngôn ngữ nhân quả một chiều (unidirectional causal).
- Chỉ sử dụng cấu trúc giải mã (decoder) so với mô hình dịch máy encoder-decoder nguyên thủy.

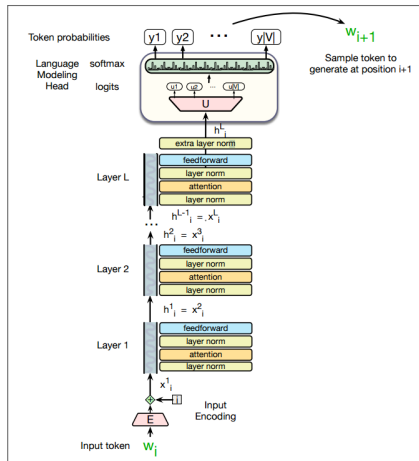


Figure 8.16 A transformer language model (decoder-only), stacking transformer blocks and mapping from an input token w_i to a predicted next token w_{i+1} .

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.6. Lấy mẫu (Sampling)

Sự cân bằng (Trade-off):

- Các phương pháp lấy mẫu luôn phải **cân bằng** giữa hai yếu tố quan trọng: **chất lượng (quality)** và **sự đa dạng (diversity)**.

Phân tích:

- **Thiên về xác suất cao:** Tạo ra văn bản **chính xác, mạch lạc** và bám sát thực tế, nhưng thường **nhàm chán và lặp lại**.
- **Thiên về xác suất trung bình:** Tăng tính **sáng tạo và đa dạng**, nhưng tiềm ẩn rủi ro **thiếu liên kết** hoặc chất lượng thấp.

8.6.1. Phương pháp Lấy mẫu Top-k

Khái niệm cốt lõi:

- Là sự **tổng quát hóa** của **Greedy Decoding**.

Thuật toán thực thi:

- 1 **Chọn trước** tham số k .
- 2 **Tính Likelihood** cho mọi từ.
- 3 Sắp xếp và **chỉ giữ lại k từ** có xác suất cao nhất.
- 4 **Chuẩn hóa lại (Renormalize)** và **lấy mẫu ngẫu nhiên** từ k từ này.

8.6.1. Phương pháp Lấy mẫu Top-k

Tokens	Logits	Scaled logits	Top-k	Softmax
chance	-107.92	-134.90	-134.90	48.30%
good	-108.63	-135.79	-135.79	19.93%
little	-108.82	-136.03	-136.03	15.64%
break	-109.35	-136.69	-136.69	8.07%
hug	-109.35	-136.69	-136.69	8.06%
shot	-109.39	-136.74	$-\infty$	0%
big	-109.43	-136.79	$-\infty$	0%
ride	-109.47	-136.84	$-\infty$	0%
free	-109.71	-137.14	$-\infty$	0%
piece	-110.13	-137.67	$-\infty$	0%
reason	-110.24	-137.80	$-\infty$	0%
heads	-110.25	-137.81	$-\infty$	0%
shout	-110.32	-137.89	$-\infty$	0%

Minh họa cắt xén phân phối bằng Top-k, với $k = 5$

8.6.2. Nucleus Sampling (Top-p) - Vấn đề của Top-k

Hạn chế của Top-k:

- Tham số k là **cố định (fixed)**, trong khi hình dạng phân phối xác suất lại **thay đổi theo ngữ cảnh**.

Giải pháp Top-p:

- Thay vì đếm số lượng từ, hệ thống giữ lại **top p phần trăm của khối lượng xác suất (probability mass)**.

8.6.2. Nucleus Sampling (Top-p) - Vấn đề của Top-k

Tokens	Logits	Scaled logits	Top-k	Softmax
chance	-107.92	-53.96	-53.96	6.93%
good	-108.63	-54.31	-54.31	4.86%
little	-108.82	-54.41	-54.41	4.42%
break	-109.35	-54.68	-54.68	3.39%
hug	-109.35	-54.68	-54.68	3.39%
shot	-109.39	-54.69	-54.69	3.33%
big	-109.43	-54.72	-54.72	3.25%
ride	-109.47	-54.74	-54.74	3.19%
free	-109.71	-54.85	-54.85	2.83%
piece	-110.13	-55.07	-55.07	2.29%



Tokens	Logits	Scaled logits	Top-k	Softmax
chance	-107.92	-53.96	-53.96	30.16%
good	-108.63	-54.31	-54.31	21.16%
little	-108.82	-54.41	-54.41	19.21%
break	-109.35	-54.68	-54.68	14.74%
hug	-109.35	-54.68	-54.68	14.73%
shot	-109.39	-54.69	-∞	0%
big	-109.43	-54.72	-∞	0%

Sự thay đổi phân phối xác suất khi $k = 50 \rightarrow k = 5$

8.6.2. Nucleus Sampling (Top-p)

Mục tiêu:

- **Cắt bỏ phần đuôi phân phối** (các từ rất ít có khả năng xảy ra).
- Giúp tập chứa ứng viên **linh hoạt tùy độ tự tin** của mô hình.

Phương trình tối ưu:

Xác định tập từ vựng $V^{(p)}$

Lựa chọn **tập hợp nhỏ nhất** thỏa mãn:

$$\sum_{w \in V^{(p)}} P(w|w_{<t}) \geq p \quad (8.48)$$

8.6.2. Nucleus Sampling (Top-p)

Tokens	Logits	Scaled logits	Softmax & Top-p	Normalization
chance	-107.92	-53.96	0.07	17.31%
good	-108.63	-54.31	0.05	12.14%
little	-108.82	-54.41	0.04	11.02%
break	-109.35	-54.68	0.03	8.46%
hug	-109.35	-54.68	0.03	8.46%
shot	-109.39	-54.69	0.03	8.31%
big	-109.43	-54.72	0.03	8.12%
ride	-109.47	-54.74	0.03	7.96%
free	-109.71	-54.85	0.03	7.08%
piece	-110.13	-55.07	0.02	5.72%
reason	-110.24	-55.12	0.02	5.43%
heads	-110.25	-55.13	0.02	0%
shout	-110.32	-55.16	0.02	0%
hard	-110.42	-55.21	0.02	0%
great	-110.42	-55.21	0.02	0%

Minh họa bao phủ khối lượng xác suất Top-p

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.7. Hàm mục tiêu Cross-Entropy

Cơ sở Đánh giá (Loss Function):

- Các mô hình ngôn ngữ lớn (LLM) được huấn luyện bằng **hàm mất mát Cross-Entropy**, hay còn gọi là **Negative Log-Likelihood**.
- Tại bước thời gian t , mức phạt là **logarit âm của xác suất** dự đoán từ đúng:

$$L_t = -\log p(w_{t+1})$$

Tối ưu hóa Trọng số:

- Mất mát của chuỗi là **trung bình cộng Cross-Entropy** trên toàn bộ chuỗi có độ dài T :

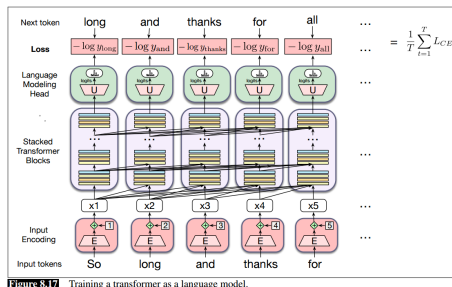
$$L = -\frac{1}{T} \sum_{t=1}^T \log p(w_t)$$

- Trọng số mạng nơ-ron được **cập nhật qua Gradient Descent** nhằm **tối thiểu hóa Loss trung bình** này.

8.7. Khả năng Huấn luyện Song song

Xử lý song song (Parallelism):

- Đầu ra cho mỗi phần tử trong chuỗi được **tính toán một cách độc lập**.
- Nhờ cấu trúc này, mỗi phần tử huấn luyện có thể được **xử lý song song**.
- Quá trình này **khắc phục hoàn toàn điểm yếu tính toán** của các mạng **RNN truyền thống**.



8.7. Quản lý Quy mô Dữ liệu (Data Scaling)

Kỹ thuật tối ưu hóa phần cứng:

- **Cửa sổ ngữ cảnh:** Các mô hình lớn được huấn luyện bằng cách lấp đầy toàn bộ cửa sổ ngữ cảnh (ví dụ: 4096 tokens cho GPT-4, 8192 cho Llama 3).
- **Data Packing:** Nếu tài liệu gốc ngắn hơn cửa sổ, nhiều tài liệu sẽ được "đóng gói" chung và ngăn cách bởi một token end-of-text đặc biệt.
- **Batch Size:** Kích thước lô dùng cho Gradient Descent thường rất lớn.

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.8.1. Scaling Laws

Hiệu năng (Loss) của các LLM phụ thuộc vào 3 yếu tố chính:

- **Model size (N):** số lượng tham số của mô hình.
- **Dataset size (D):** kích thước tập dữ liệu huấn luyện.
- **Compute (C):** tổng tài nguyên tính toán.

Mối quan hệ giữa hiệu năng và 3 yếu tố này còn được gọi là **scaling laws**.

8.8.1 Scaling Laws

Kaplan et al. (2020) đưa ra công thức về mối quan hệ giữa Loss L với số lượng tham số non-embedding N , kích thước tập dữ liệu huấn luyện D và tài nguyên tính toán C :

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (8.49)$$

$$L(D) = \left(\frac{D_c}{D} \right)^{\alpha_D} \quad (8.50)$$

$$L(C) = \left(\frac{C_c}{C} \right)^{\alpha_C} \quad (8.51)$$

8.8.1 Scaling Laws

Đối với thí nghiệm của **Kaplan et al. (2020)**, các giá trị cụ thể được xác định là:

- $\alpha_N = 0.076$, $N_c = 8.8 \times 10^{13}$ parameters
- $\alpha_D = 0.095$, $D_c = 5.4 \times 10^{13}$ tokens
- $\alpha_C = 0.050$, $C_c = 3.1 \times 10^8$ petaflop-days

8.8.1 Scaling Laws

Hoffmann et al (2022):

Approach	Coeff. a where $N_{\text{opt}} \propto C^a$	Coeff. b where $D_{\text{opt}} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

8.8.1 Scaling Laws

Số lượng tham số N có thể được ước lượng gần đúng từ các thành phần kiến trúc của Transformer:

$$\begin{aligned} N &\approx 2 d n_{\text{layer}}(2d_{\text{attn}} + d_{\text{ff}}) \\ &\approx 12 n_{\text{layer}} d^2 \\ &\text{(giả sử } d_{\text{attn}} = d_{\text{ff}}/4 = d) \end{aligned} \tag{8.52}$$

Trong đó:

- d : chiều input-output, d_{attn} : chiều self-attention, d_{ff} : chiều feedforward.
- Số tham số tăng rất nhanh khi tăng chiều rộng (d^2) và số lớp (n_{layer}).

8.8.1 Scaling Laws

Scaling laws hữu dụng trong các trường hợp sau:

- **Tối ưu hóa huấn luyện:** Xác định cách cấu hình mô hình để đạt mức performance mong muốn.
- **Dự báo hiệu năng:** Dự đoán sự thay đổi của Loss khi tăng quy mô dữ liệu hoặc tham số.
- **Cân bằng Data & Model:** Ước lượng lượng dữ liệu cần thiết khi scale-up, tránh việc tăng tham số một cách "mù quáng".
- **Tiết kiệm chi phí:** Sử dụng các thí nghiệm nhỏ để dự đoán hành vi của mô hình lớn, giảm thiểu rủi ro tài chính.

8.8.2 KV Cache

Khi huấn luyện, Attention được tính toán song song rất hiệu quả qua phép nhân ma trận:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (8.53)$$

Việc tính toán Attention trong inference (suy luận) xảy ra vấn đề gì?

8.8.2 KV Cache

Khi huấn luyện, Attention được tính toán song song rất hiệu quả qua phép nhân ma trận:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (8.53)$$

Việc tính toán Attention trong inference (suy luận) xảy ra vấn đề gì?

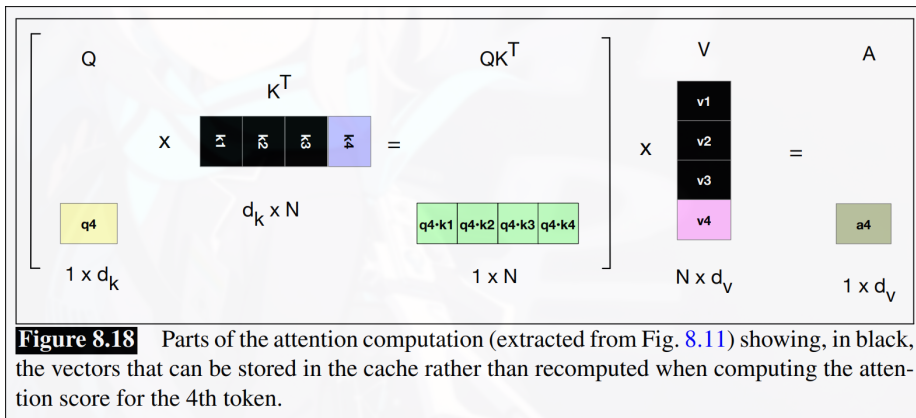
- Các token được tạo ra **tuần tự** từng cái một.
- Với mỗi token mới x_i , ta cần tính các vector Q, K, V .
- Việc tính toán lại K và V cho tất cả các token trước đó ($x_{<i}$) là cực kỳ lãng phí tài nguyên và thời gian.

8.8.2 KV Cache

KV Cache giúp tối ưu hóa quá trình suy luận (inference) bằng cách tránh các tính toán dư thừa:

- Thay vì tính toán lại từ đầu, ta sẽ **lưu trữ lại** các giá trị **Key và Value** của các token trước đó vào cache.
- Khi một token mới được sinh ra, mô hình chỉ cần thực hiện tính toán bộ ba **QKV** cho **duy nhất token đó**.
- Sau đó, nó sẽ kết hợp với các **Key và Value** đã được **lưu trong cache** để hoàn tất phép toán Attention.

8.8.2 KV Cache



8.8.3 Parameter Efficient Fine Tuning (PEFT)

Mô hình pretrained thường được finetune trên dữ liệu domain mới để học thêm kiến thức chuyên biệt.

Tuy nhiên, full finetuning trên LLM rất tốn bộ nhớ, compute và thời gian vì **số tham số quá lớn**.

⇒ Từ đó xuất hiện các phương pháp chỉ finetune **một phần nhỏ tham số** thay vì thay đổi toàn bộ, và nhóm phương pháp này được gọi là **parameter-efficient fine tuning (PERT)**.

8.8.3 Parameter Efficient Fine Tuning (PEFT)

LoRA (Low-Rank Adaptation) là một phương pháp thuộc nhóm PEFT.

Ý tưởng chính của LoRA là, thay vì cập nhật trực tiếp các ma trận lớn trong Transformer, LoRA giữ nguyên chúng và chỉ học thêm **một phần hiệu chỉnh nhỏ**. Nhờ vậy, số tham số cần huấn luyện giảm mạnh, giúp tiết kiệm bộ nhớ, compute và thời gian so với full finetuning.

8.8.3 Parameter Efficient Fine Tuning (PEFT)

Cập nhật thông thường

Giả sử ma trận cần finetune là

$$W \in \mathbb{R}^{k \times d}$$

Ta cập nhật trực tiếp:

$$\Delta W \in \mathbb{R}^{k \times d}$$

Ma trận sau khi cập nhật:

$$W + \Delta W$$

Phép biến đổi đầu ra:

$$h = xW$$

Cập nhật bằng LoRA

Đóng băng W và học hai ma trận hạng thấp:

$$A \in \mathbb{R}^{k \times r}, \quad B \in \mathbb{R}^{r \times d}$$

$$r \ll \min(k, d)$$

Thay vì học ΔW , LoRA dùng:

$$W + \Delta W \approx W + AB$$

Phép biến đổi đầu ra trở thành:

$$h = xW + xAB \quad (8.54)$$

8.8.3 Parameter Efficient Fine Tuning (PEFT)

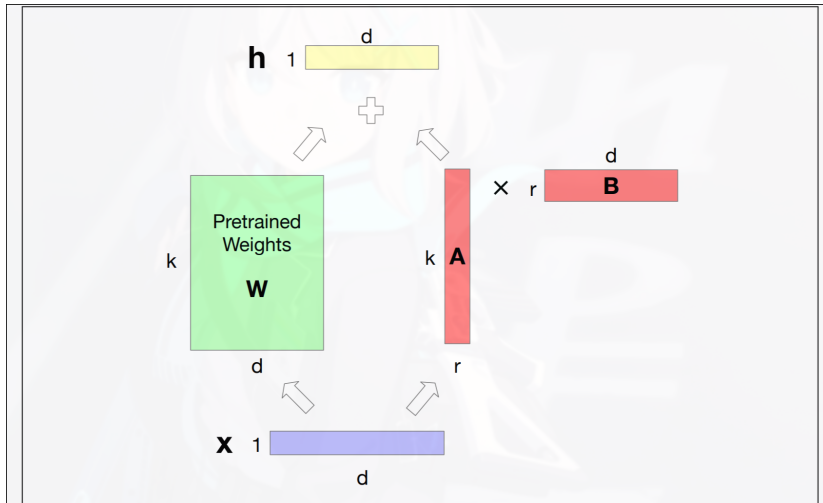


Figure 8.19 The intuition of LoRA. We freeze W to its pretrained values, and instead fine-tune by training a pair of matrices A and B , updating those instead of W , and just sum W and the updated AB .

8.8.3 Parameter Efficient Fine Tuning (PEFT)

Phương pháp	Cách làm	Ưu điểm	Nhược điểm
LoRA	Đóng băng trọng số gốc, chỉ học hai ma trận hạng thấp để biểu diễn phần cập nhật của trọng số	Hiệu quả tốt, ít tham số cần học, phổ biến, suy luận gần như không bị tăng độ trễ nhiều	Vẫn cần backbone ở precision tương đối cao nếu không kết hợp lượng tử hóa
QLoRA	Lượng tử hóa backbone xuống 4-bit rồi gắn LoRA để fine-tune	Tiết kiệm VRAM rất mạnh, phù hợp khi tài nguyên phần cứng hạn chế, vẫn giữ hiệu năng tốt	Pipeline phức tạp hơn, phụ thuộc vào quantization và thư viện hỗ trợ
Prompt Tuning	Giữ nguyên mô hình, chỉ học một số soft prompt gắn vào đầu vào để dẫn hướng mô hình	Rất ít tham số cần học, đơn giản, phù hợp với mô hình rất lớn	Thường kém ổn định hơn LoRA, hiệu quả phụ thuộc mạnh vào task và thiết kế prompt
IA3	Học các vector scale nhỏ để điều chỉnh activation trong attention và feed-forward	Rất ít tham số, hiệu quả khá tốt, gọn hơn nhiều so với fine-tune toàn bộ	Ít trực quan hơn Adapter, mức linh hoạt và phổ biến thực tế thường không bằng LoRA

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.9.1. In-Context Learning and Induction Heads

Ta có một số định nghĩa sau:

- **Pretraining:** mô hình cập nhật tham số bằng gradient descent để giảm loss.
- **Prompting:** không thay đổi trọng số, mô hình chỉ tận dụng thông tin có sẵn trong context khi suy luận.
- **In-Context Learning** có thể hiểu là khả năng mô hình học cách làm task mới ngay trong inference, mà không cần bất kỳ bước cập nhật tham số nào.

8.9.1. In-Context Learning and Induction Heads

Vậy **In-Context Learning** hoạt động như nào?

→ Cơ chế chính xác của in-context learning **vẫn chưa được hiểu hoàn toàn**, nhưng một giả thuyết nổi bật là vai trò của **induction heads**.

8.9.1. In-Context Learning and Induction Heads

Vậy **In-Context Learning** hoạt động như nào?

→ Cơ chế chính xác của in-context learning **vẫn chưa được hiểu hoàn toàn**, nhưng một giả thuyết nổi bật là vai trò của **induction heads**.

Chức năng chính của induction heads là hỗ trợ **dự đoán các pattern** lặp trong chuỗi:

- Khi gặp một token ở hiện tại, mô hình tìm về lần xuất hiện trước đó của token này.
- Sau đó nó có xu hướng “copy” token từng đi sau nó ở quá khứ để dự đoán token tiếp theo.

8.9.1. In-Context Learning and Induction Heads

Vậy **In-Context Learning** hoạt động như nào?

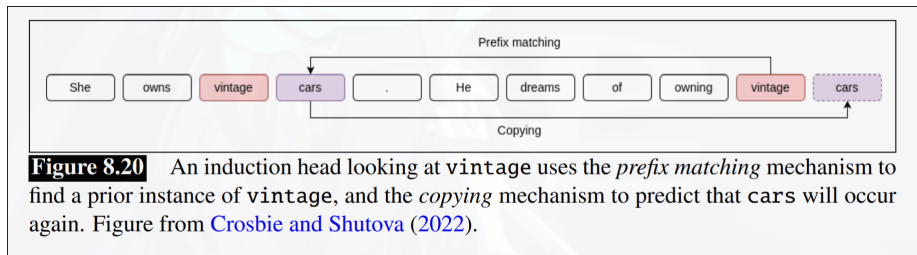
→ Cơ chế chính xác của in-context learning **vẫn chưa được hiểu hoàn toàn**, nhưng một giả thuyết nổi bật là vai trò của **induction heads**.

Chức năng chính của induction heads là hỗ trợ **dự đoán các pattern** lặp trong chuỗi:

- Khi gặp một token ở hiện tại, mô hình tìm về lần xuất hiện trước đó của token này.
- Sau đó nó có xu hướng “copy” token từng đi sau nó ở quá khứ để dự đoán token tiếp theo.

Các nghiên cứu **ablation** cho thấy khi làm yếu hoặc loại bỏ induction heads, khả năng in-context learning giảm rõ rệt. Điều này cho thấy chúng có **liên quan quan trọng**, dù chưa giải thích toàn bộ hiện tượng.

8.9.1. In-Context Learning and Induction Heads



8.9.2. Logit Lens

Mục tiêu: Quan sát quá trình hình thành dự đoán **bên trong các lớp ẩn**, thay vì chỉ nhìn output cuối cùng.

- Lấy **vector trạng thái ẩn** tại một lớp trung gian bất kỳ.
- Chiếu trực tiếp vector này lên **không gian từ vựng** bằng ma trận giải nhúng E^T .
- Giúp visualize mô hình đang "nghĩ" về từ nào ở từng giai đoạn xử lý.

8.5. The Language Modeling Head

8.6. More on Sampling

8.7. Training

8.8. Dealing with Scale

8.9. Interpreting the Transformer

8.10. Summary

8.10. Summary

- Transformer là kiến trúc không hồi quy, dựa trên multi-head self-attention để mô hình hóa quan hệ giữa các token trong ngữ cảnh.
- Một transformer block gồm residual connection, layer normalization, multi-head attention và feed-forward; nhiều block được xếp chồng để tăng năng lực biểu diễn.
- Đầu vào của transformer là tổng của token embedding và positional encoding để mô hình biết cả nội dung lẫn vị trí của token.
- Trong language model, đầu ra từ tầng cuối được chiếu qua unembedding matrix để tạo logits, rồi qua softmax để dự đoán xác suất token tiếp theo.
- Transformer-based language models có thể xử lý ngữ cảnh rất dài, nhờ đó tận dụng được nhiều thông tin hơn khi dự đoán.